

并行计算笔记整理(中)(gpt5.5 plus)

主讲：陈国良教授 原课程链接：[超算学堂EasyHPC 并行计算 - 结构·算法·编程](#)

PC7

1. 第七章总体结构
2. 7.1 PCAM 设计方法学
3. 7.2 划分
4. 7.2.2 域分解
5. 7.2.3 功能分解
6. 7.2.4 划分判据
7. 7.3 通讯
8. 7.3.2 四种通讯模式
9. 7.3.3 通讯判据
10. 7.4 组合
11. 7.4.2 表面-容积效应
12. 7.4.3 重复计算
13. 7.4.4 组合判据
14. 7.5 映射
15. 7.5.2 负载均衡算法
16. 7.5.3 任务调度算法
17. 7.5.4 映射判据
18. 7.6 小结
19. PCAM 四阶段总对比
20. 本章核心理解

PC8

1. 第八章总体：基本通讯操作
2. 8.0 预备知识
3. 8.1 选路方法与开关技术
4. 8.1.1 选路方法
5. X-Y 选路算法
6. E-Cube 选路算法
7. 8.1.2 开关技术
8. 存储转发 Store-and-Forward
9. 直通 Cut-Through
10. 虫孔 Wormhole
11. 8.2 单一信包一到一传输
12. 8.3 一到多播送
13. 8.3.1 一到多播送：SF 模式
14. 8.3.2 一到多播送：CT 模式
15. 8.4 多到多播送
16. 多到多播送：SF 模式
17. 多到多播送：CT 模式
18. 本章公式总表
19. 本章核心总结

PC9

1. 第九章总体：稠密矩阵运算
2. 9.1 矩阵的划分
3. 9.1.1 带状划分
4. 9.1.2 棋盘划分
5. 9.2 矩阵转置
6. 9.2.1 棋盘划分的矩阵转置
7. 9.2.2 带状划分的矩阵转置
8. 9.3 矩阵-向量乘法
9. 9.3.1 带状划分的矩阵-向量乘法
10. 9.3.2 棋盘划分的矩阵-向量乘法
11. 带状划分与棋盘划分比较
12. 9.4 矩阵乘法
13. 矩阵乘法并行实现方法
14. 9.4.1 简单并行分块乘法
15. 9.4.2 Cannon 乘法
16. 9.4.3 Fox 乘法
17. 9.4.4 Systolic 乘法
18. 9.4.5 DNS 乘法
19. 本章算法总对比
20. 本章核心总结

PC7

1. 第七章总体结构

第七章标题是 **并行算法的一般设计过程**。

它包括六节：

1.1 7.1 PCAM 设计方法学

介绍并行算法设计的四个阶段：

Partitioning → Communication → Agglomeration → Mapping
划分 → 通讯 → 组合 → 映射

1.2 7.2 划分

把问题分解成小任务，尽可能开拓并发性。

1.3 7.3 通讯

分析任务之间的数据交换关系。

1.4 7.4 组合

把过细的小任务合并成更大的任务，以减少通信和调度开销。

1.5 7.5 映射

把任务分配到具体处理器上，并保持负载均衡。

1.6 7.6 小结

总结 PCAM 四阶段的作用。

2. 7.1 PCAM 设计方法学

PCAM 是并行算法设计中非常经典的一套方法论。

2.1 PCAM 四个阶段

PCAM 代表四个英文词：

P: Partitioning, 划分
C: Communication, 通讯
A: Agglomeration, 组合
M: Mapping, 映射

课件第 4 页给出四个阶段的含义：

阶段	中文	作用
Partitioning	划分	分解成小任务，开拓并发性
Communication	通讯	确定任务之间的数据交换，检查划分是否合理
Agglomeration	组合	根据任务局部性，把小任务组合成更大任务
Mapping	映射	把任务分配到处理器上，提高算法性能

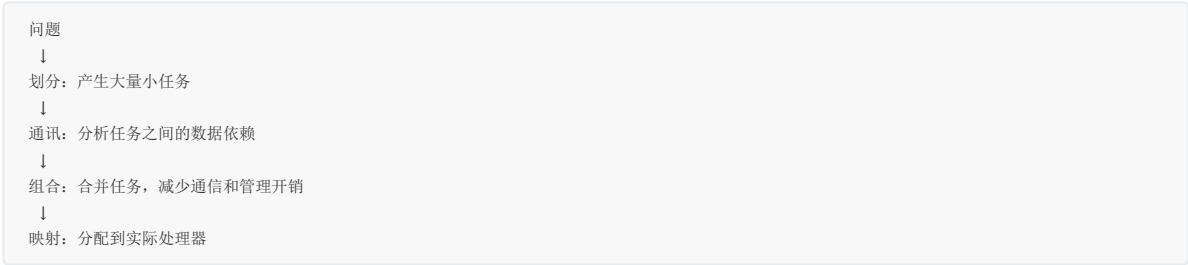
可以简单记成：

先拆开 → 看通信 → 再合并 → 放到处理器上

2.2 PCAM 的设计逻辑

PCAM 不是一开始就考虑“有多少个处理器”，而是先从问题本身出发。

它的顺序是：



第 5 页图展示了这个过程：
最开始的问题被分解成很多细粒度任务，然后任务之间形成通信图，再把多个小任务组合成较大的任务组，最后映射到处理器集合中。
这个图很重要，因为它说明：

并行算法设计不是一步到位的，而是从抽象到具体、从并发性到性能优化逐步推进。

2.3 PCAM 的核心目标

PCAM 的前两个阶段主要追求：

并发性
可扩展性

后两个阶段主要追求：

效率
局部性
负载平衡
实际性能

所以 PCAM 体现的是一种权衡：

任务越细，并发性越高，但通信和调度开销也越大；
任务越粗，通信少，但可能并发性不足。

3. 7.2 划分

划分是 PCAM 的第一步。

3.1 划分方法描述

课件第 8 页给出划分阶段的原则：

- 充分开拓算法的并发性 and 可扩放性；
- 先进行数据分解，也就是域分解；
- 再进行计算功能的分解，也就是功能分解；
- 尽量使数据集和计算集互不相交；
- 划分阶段暂时忽略处理器数目和目标机器体系结构。

这里最重要的一点是：

划分阶段不是马上考虑机器有多少处理器，而是尽可能发现问题中潜在的并行性。

也就是说，即使目标机器只有 16 个处理器，在划分阶段也可以先分出几百甚至几千个任务。后面再通过组合和映射调整任务粒度。

3.2 两类划分

课件把划分分成两类：

域分解 domain decomposition
功能分解 functional decomposition

两者的区别是：

类型	划分对象	出发点
----	------	-----

类型	划分对象	出发点
域分解	数据	把数据切成多个片段
功能分解	计算	把计算过程切成多个任务

4. 7.2.2 域分解

4.1 域分解的定义

域分解的划分对象是 **数据**。

数据可以是：

- 输入数据；
- 中间处理数据；
- 输出数据。

域分解的基本做法是：

把数据分解成大小大致相等的小数据片，每个任务负责一片数据上的计算。

例如矩阵计算中，可以把矩阵按行、按列或按块划分。

4.2 域分解时要考虑操作

域分解不能只看数据本身，还要看数据上的操作。

例如二维网格计算中，一个格点更新可能依赖上下左右邻居：

```
x[i][j] 依赖：
x[i-1][j], x[i+1][j], x[i][j-1], x[i][j+1]
```

如果把网格切成多个区域，那么区域边界上的点就需要从邻近区域获取数据。

所以课件说：

如果一个任务需要别的任务中的数据，则会产生任务间的通讯。

4.3 三维网格的域分解

课件第 11 页图 7.2 展示了三维网格的三种域分解方式：

```
1-D 分解
2-D 分解
3-D 分解
```

4.3.1 1-D 分解

只沿一个方向切分，例如把三维网格切成一层一层的 slab。

优点是简单。

缺点是当处理器数很多时，每个子域表面积相对较大，通信压力可能较高。

4.3.2 2-D 分解

沿两个方向切分，例如把三维网格切成柱状块。

相比 1-D 分解，子域更接近块状，通常通信/计算比更好。

4.3.3 3-D 分解

沿三个方向都切分，把三维网格切成小立方体。

这种方式通常更适合大规模并行，因为每个子任务体积更均衡，通信也更局部。

4.4 不规则区域的域分解

课件第 12 页展示了一个不规则网格区域的分解例子。

左边是一个复杂形状的三角网格，右边被划分成多个彩色区域。

这说明域分解不仅适用于规则矩阵或规则网格，也适用于：

- 有限元网格；
- 不规则图；
- 复杂几何模型；
- 非结构化网格。

但不规则区域分解更困难，因为要同时考虑：

任务规模均衡
边界通信尽量少
区域连通性
几何形状复杂性

5. 7.2.3 功能分解

5.1 功能分解的定义

功能分解的划分对象是 **计算**。

它不是先切数据，而是先看整个问题包含哪些不同功能模块，然后把这些功能模块划分成不同任务。

例如一个气候模拟程序可能包括：

大气模型
海洋模型
水文模型
陆面模型

这些模块之间相互交换数据，但每个模块内部的计算逻辑不同。

5.2 功能分解和域分解的区别

域分解问的是：

数据怎么切？

功能分解问的是：

计算流程怎么拆？

例如图像处理任务可以功能分解为：

读取图像 → 去噪 → 边缘检测 → 特征提取 → 输出结果

每个阶段是不同功能。

5.3 功能分解的判断

课件第 14 页指出：

划分后，要研究不同任务所需的数据。

如果这些数据不相交，说明功能分解比较成功。

如果不同任务需要大量重叠数据，就说明功能分解可能不合理，需要重新进行域分解和功能分解。

这句话非常关键：

功能分解不是只看功能模块是否不同，还要看不同功能模块的数据依赖是否会导致严重通信。

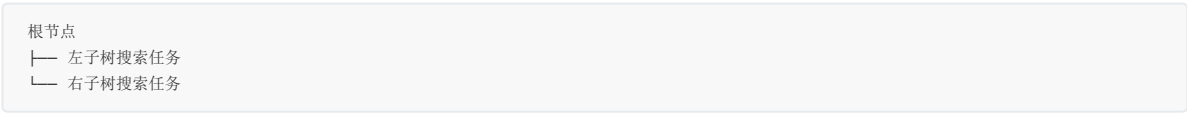
5.4 功能分解例子

课件第 15 页给出两个例子。

5.4.1 搜索树

搜索树中不同分支可以看成不同任务。

例如：

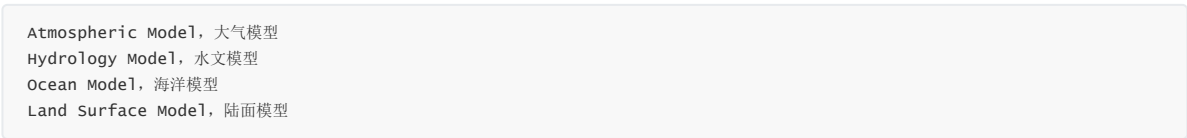


如果不同分支之间相对独立，就可以并行搜索。

但如果有共享剪枝信息或全局最优值，就会产生通信和同步。

5.4.2 气候模型

图中展示了多个模型模块：



这些模块之间存在数据交换。

例如：

- 大气模型需要海洋温度；
- 海洋模型需要风场；
- 陆面模型需要降雨量；
- 水文模型需要地表径流。

这就是典型的功能分解。

6. 7.2.4 划分判据

课件第 17 页给出划分阶段的检查标准。

6.1 划分是否具有灵活性

好的划分应该方便后续组合和映射。

如果一开始划得太死，后面就难以调整任务粒度。

6.2 是否避免冗余计算和存储

划分后，如果多个任务重复计算大量相同内容，或者重复存储大量数据，就可能浪费资源。

但注意：后面组合阶段会讲，有时适当重复计算可以减少通信。

所以这里强调的是不要在划分阶段产生不必要的冗余。

6.3 任务尺寸是否大致相当

任务太不均衡会导致负载不平衡。

例如一个任务计算 1 秒，另一个任务计算 100 秒，那么快的处理器会长时间等待慢的处理器。

6.4 任务数是否与问题尺寸成比例

当问题规模变大时，任务数也应该随之增加。

否则算法不可扩展。

例如问题规模扩大 100 倍，但任务数仍然固定，那么无法利用更多处理器。

6.5 功能分解是否合理

功能分解是一种更深层次的分解，需要判断不同功能之间的数据重叠是否过多。

如果功能之间耦合过强，就不适合简单按功能拆分。

7. 7.3 通讯

通讯是 PCAM 的第二步。

7.1 通讯方法描述

课件第 20 页指出：

- 通讯是 PCAM 设计过程的重要阶段；
- 划分产生的任务一般不能完全独立执行；
- 任务之间需要数据交流，因此产生通讯；
- 功能分解确定任务之间的数据流；
- 任务是并发执行的，但通讯会限制这种并发性。

这说明：

并行算法不是任务越多越好，关键还要看任务之间要通信多少。

通信太多时，并行性能会被严重限制。

7.2 通讯的作用

通讯阶段有两个作用：

7.2.1 描述数据依赖

明确哪些任务需要从哪些任务获得数据。

例如：

任务 A 计算边界值
任务 B 需要 A 的边界值

那么 A 和 B 之间存在通信。

7.2.2 检查划分是否合理

如果发现划分后任务之间通信非常复杂，说明前面的划分可能不好，需要重新划分。

所以 PCAM 不是线性流程，而是可能反复迭代：

划分 → 通讯分析 → 发现通信太大 → 回去重新划分

8. 7.3.2 四种通讯模式

课件第 22 页列出四种通信模式分类：

局部 / 全局通讯
结构化 / 非结构化通讯
静态 / 动态通讯
同步 / 异步通讯

8.1 局部通讯

局部通讯是指通讯限制在一个邻域内。

课件第 23 页图中，中心节点只和上下左右邻居通信。

典型例子：

二维网格五点差分
三维网格七点差分
局部图更新
细胞自动机

局部通讯通常比较容易优化，因为通信对象固定、距离近、模式规则。

8.2 全局通讯

全局通讯是非局部的。

课件第 24 页给出两个例子：

All-to-All
Master-Worker

8.2.1 All-to-All

每个任务都可能和所有其他任务通信。

例如并行转置、全交换、某些排序算法中的数据重分布。

特点是通信量大，容易产生网络拥塞。

8.2.2 Master-Worker

一个主任务负责调度，多个工作任务负责执行。

优点是结构简单，适合动态任务分配。

缺点是 master 容易成为瓶颈。

8.3 结构化通讯

结构化通讯是指：

每个任务的通讯模式是相同的。

课件第 25 页左图展示了一个规则网格，每个点和固定方向邻居通信。

例如二维网格中，每个内部点都和上下左右通信：

上
下
左
右

这种通信模式非常适合并行计算，因为：

- 通信对象规则；
- 容易预测；
- 容易优化；
- 适合静态调度。

8.4 非结构化通讯

非结构化通讯是指：

没有统一的通讯模式。

课件第 26 页给出无结构化网格作为例子。

不规则网格中，每个节点的邻居数和邻居位置都可能不同。

这类通信常见于：

- 有限元方法；
- 非结构化网格流体计算；
- 图算法；
- 社交网络分析；
- 稀疏矩阵计算。

难点是：

通信对象不规则
消息大小不均衡
负载难平衡
缓存局部性差

8.5 静态通讯

静态通讯是指通信模式在程序执行前就能确定。

例如规则网格迭代：

每轮都和上下左右邻居交换边界

优点是可以预先优化通信路径和缓冲区。

8.6 动态通讯

动态通讯是指通信模式在运行时才能确定。

例如搜索算法中，哪个任务产生新分支、任务发送到哪里，都可能动态变化。

动态通讯更灵活，但调度和性能分析更困难。

8.7 同步通讯

同步通讯中，通信双方需要在某个时间点协调。

例如发送方等待接收方，或所有任务进入 barrier 后再继续。

优点是程序逻辑清楚。

缺点是等待开销大。

8.8 异步通讯

异步通讯允许通信和计算重叠。

例如任务发送消息后继续计算，不必立即等待对方接收。

优点是可能隐藏通信延迟。

缺点是程序正确性和调试更复杂。

9. 7.3.3 通讯判据

课件第 28 页给出通信阶段的检查标准。

9.1 所有任务是否执行大致相当的通讯

如果某些任务通信量特别大，它们可能成为瓶颈。

例如 master-worker 模型中，master 的通信量往往远大于 worker。

9.2 是否尽可能局部通讯

局部通讯通常比全局通讯更容易扩展。

所以设计并行算法时，应尽量让任务只和邻居交换数据。

9.3 通讯操作是否能并行执行

如果所有任务都要排队通过同一个通信瓶颈，性能会很差。

好的通信设计应允许多个通信同时发生。

9.4 同步任务的计算能否并行执行

如果同步点太多，任务会频繁等待。

设计时要检查：

能否减少 barrier？
能否把通信和计算重叠？
能否改成异步通信？

10. 7.4 组合

组合是 PCAM 的第三步，也叫 Agglomeration。

10.1 方法描述

课件第 31 页说明：

组合是从抽象到具体的过程，目标是让组合后的任务能在某类并行机上有效执行。

它主要做几件事：

- 合并小尺寸任务，减少任务数；
- 如果任务数恰好等于处理器数，则组合也完成了映射；
- 通过增加任务粒度和重复计算，减少通信成本；
- 保持映射和扩展灵活性；
- 降低软件工程成本。

可以理解为：

划分阶段故意把任务切得很细，组合阶段再把任务合并到合适粒度。

10.2 为什么需要组合

如果任务太细，会有很多问题：

任务数量太多
调度成本太高
通信次数太多
同步太频繁
软件实现复杂

组合可以减少这些开销。

但组合过度也不好，因为会减少并发性。

所以组合阶段的核心是找平衡。

11. 7.4.2 表面-容积效应

11.1 基本概念

课件第 33 页给出重要结论：

通讯量与任务子集的表面成正比，计算量与任务子集的体积成正比。

这就是 **表面-容积效应**。

对于一个二维网格块：

计算量 $\approx$ 面积
通信量 $\approx$ 边界长度

对于一个三维网格块：

计算量 $\approx$ 体积
通信量 $\approx$ 表面积

11.2 为什么任务变大通信比例会下降

假设二维块大小是 $k \times k$ 。

计算量约为：

$$k^2$$

边界通信量约为：

$$4k$$

通信/计算比约为：

$$4k / k^2 = 4/k$$

当 k 越大，通信/计算比越小。

所以把小任务组合成大任务，通常可以降低相对通信开销。

11.3 图中含义

课件第 33 页图中，大网格被划分为若干子区域。

每个子区域内部点主要进行本地计算，边界点需要和邻居通信。

这说明：

内部点贡献计算
边界点贡献通信

任务块越大，内部点比例越高，边界点比例越低。

12. 7.4.3 重复计算

12.1 重复计算的思想

课件第 35 页指出：

重复计算可以减少通讯量，但增加了计算量，应保持恰当平衡。

也就是说，有时与其通信拿数据，不如每个任务自己重复算一遍。

这在通信代价很高、计算代价较低时很有用。

12.2 重复计算的目标

重复计算不是为了炫技，而是为了：

减少算法总运行时间

如果重复计算增加的计算时间小于节省的通信时间，那么它就是值得的。

12.3 二叉树求全和

课件第 36 页例子：

在二叉树上， N 个处理器求 N 个数的全和，并要求每个处理器都保存全和。

普通做法是两阶段：

向上归约求和
向下广播全和

需要：

$2 \log N$ 步

也就是：

- $\log N$ 步把和收集到根；
- $\log N$ 步把总和广播回所有处理器。

12.4 蝶式结构求和

课件第 37 页展示蝶式结构。

蝶式结构使用重复计算，让每个处理器在每一轮和另一个处理器交换部分和，并自己更新。

经过：

$\log N$ 步

每个处理器都得到全和。

它比二叉树方式少了一半步数。

代价是某些部分和在多个处理器中重复计算和保存。

这说明：

适当重复计算可以减少通信轮数，从而降低总时间。

13. 7.4.4 组合判据

课件第 39 页给出组合阶段的检查标准。

13.1 增加粒度是否减少了通讯成本

组合任务后，通信次数和通信边界是否真的减少？

如果任务变大但通信没少，就不值得。

13.2 重复计算是否已权衡其得益

重复计算是否真的减少总运行时间？

需要比较：

增加的计算时间
节省的通信时间

13.3 是否保持灵活性和可扩放性

组合后不能把任务数变得太少。

如果任务数太少，就无法使用更多处理器。

13.4 组合任务数是否与问题尺寸成比例

随着问题规模变大，组合后的任务数也应增加。

否则算法不可扩展。

13.5 是否保持类似的计算和通讯

组合后的任务之间应尽量负载均衡。

如果某些组合任务很大，某些很小，就会造成处理器等待。

13.6 有没有减少并行执行机会

组合会减少任务数，所以要检查是否牺牲了太多并发性。

14. 7.5 映射

映射是 PCAM 的最后一步。

14.1 方法描述

课件第 42 页指出：

每个任务要映射到具体处理器，定位到运行机器上。

当任务数大于处理器数时，就存在：

负载平衡
任务调度

问题。

14.2 映射目标

映射的目标是：

减少算法执行时间

课件给出两个原则：

14.2.1 并发任务放到不同处理器

如果两个任务可以同时执行，就尽量放在不同处理器上。

这样可以提高并行度。

14.2.2 高通讯任务放到同一处理器

如果两个任务之间通信非常频繁，可以考虑放在同一处理器上。

这样可以减少通信成本。

14.3 映射的本质是权衡

这两个原则可能冲突。

例如任务 A 和 B 可以并发执行，但它们之间通信也很多。

如果放到不同处理器：

并发性好，但通信成本高

如果放到同一处理器：

通信成本低，但失去并发性

所以映射是权衡问题。

课件指出，映射属于 **NP 完全问题**。

这意味着一般情况下很难找到最优映射，只能用启发式方法。

15. 7.5.2 负载均衡算法

课件第 44 页列出几类负载均衡算法。

15.1 静态负载均衡

静态方法是在程序执行前就确定任务分配。

适合：

- 任务规模可预测；
- 通信模式固定；
- 数据分布规则。

优点是调度开销低。

缺点是不能适应运行时变化。

15.2 概率负载均衡

概率方法通过随机方式分配任务。

优点是简单，有时能避免最坏情况。

例如把任务随机分给处理器，期望上负载较均衡。

缺点是结果不确定，且不一定适合通信密集任务。

15.3 动态负载均衡

动态方法在程序执行期间根据实际负载调整任务分配。

适合：

- 任务运行时间不可预测；
- 搜索问题；
- 自适应网格；
- 不规则计算。

缺点是调度成本较高。

15.4 基于域分解的负载均衡方法

课件列出四种：

递归对剖
局部算法
概率方法
循环映射

15.4.1 递归对剖

把区域不断一分为二，使每部分负载尽量相等。

适合几何区域或网格划分。

15.4.2 局部算法

通过局部调整把负载从重载区域迁移到轻载区域。

适合动态负载变化。

15.4.3 概率方法

用随机化方法分配任务，降低负载倾斜概率。

15.4.4 循环映射

按轮转方式把任务分配给处理器。

例如：

任务1 → p1
任务2 → p2
...
任务p → pp
任务p+1 → p1

适合任务数量多、任务代价差异不太大的情况。

16. 7.5.3 任务调度算法

16.1 基本思想

课件第 46 页说明：

任务可以放在集中的或分散的任务池中，然后用调度算法把任务分配给处理器。

这类方法适合任务动态产生或任务执行时间不确定的情况。

16.2 经理/雇员模式

经理/雇员模式也叫 master-worker。

结构是：

经理进程：管理任务池，分配任务
雇员进程：领取任务，执行任务，返回结果

优点：

- 实现简单；
- 动态负载平衡效果好；
- 适合任务粒度不均匀的场景。

缺点：

- 经理进程可能成为通信瓶颈；
- 任务太细时调度成本高；
- 扩展到大量处理器时可能性能下降。

16.3 非集中模式

非集中模式没有单一 master。

任务池可以分散在多个处理器上。

处理器之间可以进行任务窃取或局部调度。

优点：

- 没有单点瓶颈；
- 可扩展性更好。

缺点：

- 实现复杂；
- 需要处理任务发现、迁移、同步等问题。

17. 7.5.4 映射判据

课件第 48 页给出两个映射阶段的检查问题。

17.1 集中式负载均衡是否存在通信瓶颈

如果使用 master-worker 或集中任务池，需要检查 master 是否会成为瓶颈。

尤其当处理器很多、任务很细时，这个问题更严重。

17.2 动态负载均衡的调度成本如何

动态调度能改善负载均衡，但会带来额外成本。

需要判断：

动态调度节省的等待时间
是否大于
调度和通信开销

如果调度成本太高，动态负载均衡反而会降低性能。

18. 7.6 小结

课件第 50 页总结了四个阶段。

18.1 划分

划分包括：

域分解
功能分解

目标是开拓并发性。

18.2 通讯

通讯关注任务之间的数据交换。

它用于检查任务之间的数据依赖和通信代价。

18.3 组合

组合通过合并任务，使算法更有效。

核心目标是减少通信、减少任务管理开销，同时保留足够并行性。

18.4 映射

映射把任务分配到处理器，并保持负载平衡。

最终目标是降低执行时间，提高实际性能。

19. PCAM 四阶段总对比

阶段	主要问题	关注重点	常见风险
划分	如何把问题拆开	并发性、可扩展性	任务太少或负载不均
通讯	任务之间如何交换数据	通信量、通信模式	通信过多限制并发
组合	如何调整任务粒度	局部性、减少通信	并发性下降
映射	任务放到哪个处理器	负载平衡、执行时间	NP 完全，难找最优

20. 本章核心理解

第七章的主线可以总结为：

划分：尽量多地发现并行性
通讯：看这些并行任务之间是否需要交换数据
组合：把太细的任务合并到合适粒度
映射：把任务放到处理器上，并尽量负载均衡

一句话概括：

PCAM 是从问题到并行程序的系统化设计流程，它先追求并发性，再逐步考虑通信、粒度、局部性和负载平衡，最终得到能在实际并行机上运行的算法结构。

PC8

1. 第八章总体：基本通讯操作

这份 **PC8.pdf** 开始进入第三篇 **并行数值算法**。第三篇包括第八章到第十一章：基本通讯操作、稠密矩阵运算、线性方程组求解、快速傅里叶变换。本次课件讲的是 **第八章：基本通讯操作**，重点是并行机网络中的路由、开关技术，以及常见通信模式的时间分析。

1.1 本章结构

第八章包括：

8.0 预备知识
8.1 选路方法与开关技术
8.2 单一信包一到一传输
8.3 一到多播送
8.4 多到多播送

这一章的核心问题是：

在并行计算中，处理器之间如何传消息？消息经过网络需要多少时间？不同网络拓扑和开关方式会怎样影响通信代价？

2. 8.0 预备知识

2.1 选路 Routing

选路，也叫选径或路由，指的是：

确定消息从源节点到目的节点所经过的路径。

好的选路算法需要满足：

通信延迟低
避免死锁
具有一定容错能力

在并行计算中，处理器之间的数据交换都要依赖选路。

例如在二维网格中，从 (2, 1) 到 (7, 6)，可以先横向再纵向，也可以先纵向再横向；不同路径可能导致不同通信延迟和拥塞。

2.2 消息、包、片

课件区分了三个层次：

2.2.1 消息 Message

消息是多计算机系统中处理节点之间传递的信息逻辑单位。

它可以包含：

数据
同步信息
控制信息

消息是逻辑单位，可以由任意数量的包组成。

2.2.2 包 Packet

包是信息传送的基本单位。

课件中说包长度随协议不同而不同，一般为：

64 ~ 512 位

一个消息可以拆成多个包传输。

2.2.3 片 Flit

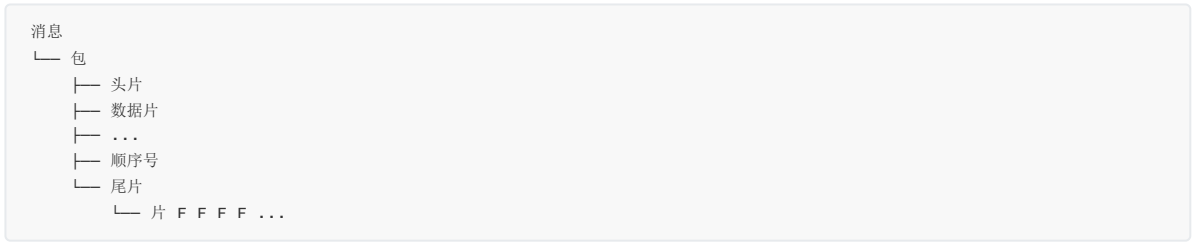
片是比包更小的固定长度单位，一般为：

8 位

在虫孔路由中，消息会被拆成多个 flit，头片负责引导路径，数据片跟随头片前进，尾片表示消息结束。

2.3 消息、包、片的关系

第 5 页图展示了三者关系：



可以这样理解：

消息是整体内容
包是消息分割后的传输单位
片是包内部更细的流水传输单位

这为后面讲存储转发、切通、虫孔路由做准备。

2.4 常用通信术语

2.4.1 信道带宽

信道带宽记为：

$$b = w \cdot f \text{ bits/sec}$$

其中：

- w 是信道宽度，即每次传输多少位；
- $f = 1/t$ 是信号传输率；
- t 是时钟周期。

如果信道宽度越大、频率越高，带宽越大。

2.4.2 节点和开关的度

节点或开关的度是：

与该节点或开关相连的信道数目。

例如二维 Mesh 内部节点通常有 4 个邻居，所以度为 4。

2.4.3 路径

路径是一个包在网络中经过的：

开关 + 链路

序列。

2.4.4 路由长度或距离

路由长度，也叫距离，是路径中包含的链路数量。

如果消息从源到目的经过 1 条链路，则路由长度为 1。

后面所有通信时间公式都和 1 有关。

2.5 信包传输性能参数

课件定义了几个重要参数：

参数	含义
<code>t_s</code>	startup time，启动时间，用于准备包头信息等
<code>t_h</code>	per-hop time，节点延迟时间，即包头穿越相邻节点的时间
<code>t_w</code>	transfer time，字传输时间，即传输每个字的时间
<code>l</code>	链路数，也就是路由距离
<code>m</code>	信包大小，通常表示消息中有多少个字

这些参数会出现在后面的通信公式中。

2.6 选路算法的三种机制

课件第 7 页讲了三种选路机制。

2.6.1 基于算术的选路

开关中具有简单算术运算功能。

例如维序选路中，开关根据当前节点和目标节点坐标判断下一步该走哪个方向。

典型例子：

二维 Mesh 中的 X-Y 选路
超立方中的 E-Cube 选路

2.6.2 基于源地址的选路

在源节点就提前算好整条路径。

然后把路径上每个开关的输出端口地址：

`p0, p1, ..., pn`

放入包头。

消息经过每个开关时，该开关只需要从包头取出对应端口信息即可。

优点是中间开关简单。

缺点是包头可能较长，而且源节点需要提前知道完整路径。

2.6.3 基于查表的选路

每个开关内部保存一个路由表。

开关根据包头中的选路域查表，确定输出端口。

优点是灵活，适合复杂网络。

缺点是需要维护路由表。

2.7 选路方式分类

课件第 8 页把选路方式分成多类。

2.7.1 按开关方式分

包括：

存储-转发 Store and Forward
虫孔 Wormhole
线路交换

其中本章重点讲存储转发、切通和虫孔。

2.7.2 按消息到达时机分

包括：

静态选路
动态选路

静态选路：选路开始时所有信息都已到达网络。

动态选路：信包可以在任意时刻到达网络。

2.7.3 按路径是否预先计算分

包括：

脱机 offline
联机 online

脱机：事先算好传输路径。

联机：没有事先计算好的路径，运行时确定路径。

2.7.4 按通信关系分

包括：

一到一：单播
一到一：置换
多到一：集中
一到多：多播
一到所有：广播、组播
多到多：会议

这些通信模式后面都会出现。

3. 8.1 选路方法与开关技术

这一节分成两部分：

8.1.1 选路方法
8.1.2 开关技术

4. 8.1.1 选路方法

4.1 选路方法分类

课件第 11 页给出两组分类。

4.1.1 最短路径与非最短路径

最短路径选路总是选择链路数最少的路径。

非最短路径可能绕路，通常用于避开拥塞或故障。

课件举例：

维序选路是贪心的最短路径选路
二阶段维序选路是随机选路

4.1.2 确定选路与自适应选路

确定选路：

路径只由源地址和目的地址决定。

自适应选路：

路径会根据当前网络状况变化。

确定选路简单、容易分析。

自适应选路更灵活，但实现复杂。

4.2 维序选路 Dimension-Ordered Routing

维序选路是一种：

确定的最短路径选路

它的思想是：

按某个固定维度顺序依次消除源地址和目的地址之间的差异。

典型例子：

二维网孔中的 X-Y 选路
超立方中的 E-Cube 选路

5. X-Y 选路算法

5.1 基本思想

X-Y 选路用于二维 Mesh 或二维 Torus。

算法很简单：

step 1: 沿 X 方向将信包送到目的处理器所在的列；
step 2: 沿 Y 方向将信包送到目的处理器所在的行。

也就是：

先横向，再纵向

5.2 例子

第 13 页图 8.1 是一个 8×8 网格。

图中给了多个源-目的对，例如：

$(2,1) \rightarrow (7,6)$
 $(0,7) \rightarrow (4,2)$
 $(5,4) \rightarrow (2,0)$
 $(6,3) \rightarrow (1,5)$

以 $(2,1) \rightarrow (7,6)$ 为例：

先沿 X 方向：

$(2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (6,1) \rightarrow (7,1)$

再沿 Y 方向：

$(7,1) \rightarrow (7,2) \rightarrow (7,3) \rightarrow (7,4) \rightarrow (7,5) \rightarrow (7,6)$

5.3 X-Y 选路特点

优点：

简单
确定
路径最短
容易硬件实现

缺点：

不考虑网络拥塞
容易在某些行或列形成热点
不够自适应

6. E-Cube 选路算法

6.1 适用网络

E-Cube 选路用于超立方网络。

在超立方中，每个节点用二进制地址表示。

如果两个节点地址只有一位不同，它们之间有一条直接链路。

6.2 路由值计算

设源地址为：

$$S = s_{\{n-1\}}s_{\{n-2\}}\dots s_{1s_0}$$

目的地址为：

$$D = d_{\{n-1\}}d_{\{n-2\}}\dots d_{1d_0}$$

计算：

$$R = S \text{ XOR } D$$

也就是：

$$r_i = s_i \oplus d_i$$

如果 $r_i = 1$ ，说明第 i 位不同，需要沿第 i 维走一步。

如果 $r_i = 0$ ，说明该维度已经一致，不需要改变。

6.3 路由过程

E-Cube 选路逐位修改源地址，使其最终变成目的地址。

每次只翻转一个不同的 bit。

路径长度等于源地址和目的地址的 Hamming distance。

6.4 例子

课件第 15 页给出：

源： $S = 0110$
目的： $D = 1101$

计算：

```
0110
XOR 1101
=   1011
```

说明第 0、1、3 位不同，需要沿这些维度移动。

课件给出的路径之一是：

```
0110 → 0111 → 0101 → 1101
```

每一步只改变一个 bit。

6.5 E-Cube 特点

优点：

```
适合超立方
路径简单
确定性强
路径长度等于二进制地址差异位数
```

缺点：

```
不考虑拥塞
不是自适应选路
```

7. 8.1.2 开关技术

这一节讲三种开关方式：

```
存储转发 Store-and-Forward
切通 Cut-Through
虫孔 wormhole
```

它们的区别在于：消息经过中间节点时，是完整存下来再转发，还是像流水线一样边到边转。

8. 存储转发 Store-and-Forward

8.1 基本过程

存储转发方式中，消息被分成信包，每个信包都含有寻径信息。

当一个信包到达中间节点 A 时：

1. A 把整个信包放入通信缓冲器；
2. A 根据选路算法选择下一个节点 B；
3. 如果 A 到 B 的通道空闲，且 B 的缓冲器可用；
4. A 把完整信包发给 B。

关键点是：

每个中间节点必须先接收完整信包，再转发。

8.2 通信时间

课件给出：

```
t_comm(SF) = t_s + (m t_w + t_h) l
```

数量级为：

$O(m+1)$

其中：

- $m \cdot t_w$ 是传完整包的时间；
- t_h 是每跳节点延迟；
- l 是路径链路数。

8.3 缺点

存储转发有两个主要缺点：

8.3.1 缓冲需求大

每个中间节点必须能缓存整个消息或整个信包。

8.3.2 延迟随跳数线性增加

因为每一跳都要完整接收再完整发送，所以网络时延与经过节点数成正比。

9. 切通 Cut-Through

9.1 基本过程

切通方式中，在传递消息之前，先建立一条从源节点到目的节点的物理通道。

在消息传递过程中：

整条路径的这一段链路都被占用
直到消息尾部经过后，物理通道才释放

与存储转发不同，切通不需要每个中间节点缓存完整消息。

9.2 通信时间

课件给出：

$$t_{\text{comm}}(\text{CT}) = t_s + m \cdot t_w + l \cdot t_h$$

数量级为：

$O(m+1)$

相比存储转发的 $O(m+1)$ ，切通显著降低了多跳长消息的延迟。

9.3 缺点

切通的缺点是：

物理通道非共享
传输过程中整条物理通道一直被占用

也就是说，如果通道被一个消息占住，其他消息不能使用这条路径。

10. 虫孔 Wormhole

10.1 基本思想

虫孔路由由 Dally 于 1986 年提出。

它结合了存储转发和切通的优点：

减少缓冲区需求

提高物理通道利用率

降低网络延迟

10.2 消息切分

虫孔路由把一个消息拆成很多很小的片：

头片

数据片

尾片

其中：

- 头片包含所有寻径信息；
- 数据片携带数据；
- 尾片包含消息结束符。

10.3 传输方式

头片在网络中向前寻找路径。
后面的数据片像虫子的身体一样跟随头片前进。

课件形象地说：

每个消息中的片以流水的方式在网络中向前“蠕动”。

当尾片离开某节点后，该节点占用的资源就被释放。

10.4 优点

课件第 20 页列出虫孔选路的优点：

10.4.1 缓冲需求小

每个节点只需要缓冲一个片，而不是整个包或整个消息。

这使它易于用 VLSI 实现。

10.4.2 网络传输延迟低

存储转发延迟与路径距离强相关。
虫孔路由在消息较长时，传输距离对延迟影响较小。

10.4.3 通道共享性好

相比线路交换，虫孔路由的通道利用率更高。

10.4.4 容易实现多播和广播

因为消息像流水一样前进，适合扩展到 multicast 和 broadcast。

10.5 三种开关方式对比

方式	中间节点是否存完整包	时间复杂度	主要特点
Store-and-Forward	是	$O(m^2)$	简单，但缓冲大、延迟高
Cut-Through	否	$O(m + 1)$	延迟低，但路径被独占
Wormhole	否，只存 flit	低延迟	缓冲小、通道利用率高

第 21 页的时空图也说明：存储转发中每一跳都要等待完整包；切通中消息像流水一样穿过多个节点。

11.8.2 单一信包一到一传输

这一节分析一个信包从一个源节点传到一个目的节点的通信时间。

11.1 距离 l 的计算

课件考虑 p 个处理器。

11.1.1 一维环形网络

最远距离为：

$$l \leq \lfloor p / 2 \rfloor$$

因为环可以双向走，最远也只需要走半圈。

11.1.2 带环绕 Mesh

对于带环绕 Mesh，最远距离为：

$$l \leq 2\lfloor \sqrt{p} / 2 \rfloor$$

因为二维环绕网格中，X 方向最多走半边，Y 方向最多走半边。

11.1.3 超立方

对于超立方：

$$l \leq \log p$$

因为 p 个节点的超立方维度是 $\log p$ 。

11.2 存储转发 SF 的一到一通信时间

由存储转发公式：

$$t_{\text{comm}}(\text{SF}) = t_s + (m t_w + t_h) l$$

课件给出简化结果。

11.2.1 一维环形

$$t_{\text{comm}}(\text{SF}) = t_s + t_w m \lfloor p/2 \rfloor$$

11.2.2 带环绕 Mesh

$$t_{\text{comm}}(\text{SF}) = t_s + 2 t_w m \lfloor \sqrt{p}/2 \rfloor$$

11.2.3 超立方

$$t_{\text{comm}}(\text{SF}) = t_s + t_w m \log p$$

这些公式说明：

在 SF 模式下，通信时间随路径长度增长。

11.3 切通 CT 的一到一通信时间

课件给出：

$$t_{\text{comm}}(\text{CT}) = t_s + m t_w$$

这里相当于忽略了每跳延迟 t_h ，或者认为主导项是消息长度。

所以在 CT 中，只要路径建立好，多跳带来的额外代价不明显。

11.4 当消息很长时

课件指出，如果：

$$m \gg p$$

则：

$$t_{\text{comm}}(\text{SF}) \approx t_{\text{comm}}(\text{CT}) = t_s + m t_w$$

因为消息长度很大时，传输数据本身的时间占主导，路径跳数差异相对不明显。

12. 8.3 一到多播送

一到多播送，也就是 one-to-all broadcast：

一个源处理器把同一个消息发送给所有处理器。

课件分别讨论 SF 模式和 CT 模式。

13. 8.3.1 一到多播送：SF 模式

13.1 环上的一到多播送

13.1.1 步骤

课件第 26 页给出环上的方法：

1. 先向左右邻近节点传送；
2. 再沿左右两个方向同时播送。

从源节点 0 出发，消息同时向两个方向扩散。

13.1.2 通信时间

$$t_{\text{one-to-all}}(\text{SF}) = (t_s + m t_w) \lceil p/2 \rceil$$

因为最远节点需要大约半圈距离。

13.2 环绕网孔上的一到多播送

13.2.1 步骤

课件第 27 页给出方法：

1. 先完成一行中的播送；
2. 再同时进行各列的播送。

第 27 页图 8.6 中，一个 4×4 网孔需要 4 步：

- 2 步行播送
- 2 步列播送

13.2.2 通信时间

$$t_{\text{one-to-all}}(\text{SF}) = 2(t_s + m t_w) \lceil \sqrt{p}/2 \rceil$$

其中 $\lceil \sqrt{p} \rceil$ 是每行或每列的处理器数。

13.3 超立方上的一到多播送

13.3.1 步骤

课件第 28 页给出方法：

从低维到高维，依次进行播送

每一维通信一次，已经收到消息的节点在下一维继续扩散。

例如 3 维超立方中：

第 1 步：1 个节点变 2 个节点
第 2 步：2 个节点变 4 个节点
第 3 步：4 个节点变 8 个节点

13.3.2 通信时间

$$t_{\text{one-to-all}}(\text{SF}) = (t_s + m t_w) \log p$$

因为 p 个节点的超立方维数是 $\log p$ 。

14. 8.3.2 一到多播送：CT 模式

14.1 环上的 CT 播送

14.1.1 步骤

课件第 30 页给出环上的 CT 播送策略：

1. 先发送到 $p/2$ 远的处理器；
2. 再同时发送到 $p/2^2$ 远的处理器；
- ...
- i. 再同时发送到 $p/2^i$ 远的处理器。

这个过程类似二分扩散。

14.1.2 通信时间

课件给出：

$$t_{\text{one-to-all}}(\text{CT}) = \sum_{i=1}^{\lceil \log p \rceil} (t_s + m t_w + t_h p/2^i)$$

化简为：

$$t_{\text{one-to-all}}(\text{CT}) = t_s \log p + m t_w \log p + t_h(p - 1)$$

如果 t_h 可忽略，则近似为：

$$t_{\text{one-to-all}}(\text{CT}) \approx (t_s + m t_w) \log p$$

14.2 网孔上的 CT 播送

14.2.1 步骤

课件第 31 页给出：

1. 先进行行播送；
2. 再同时进行列播送。

14.2.2 通信时间

行播送和列播送各自类似一维二分扩散。

课件给出：

$$\begin{aligned} t_{\text{one-to-all}}(\text{CT}) \\ = 2(t_s \log p + m t_w \log p + t_h(\sqrt{p} - 1)) \end{aligned}$$

化简为：

$$\begin{aligned} t_{\text{one-to-all}}(\text{CT}) \\ = (t_s + m t_w) \log p + 2t_h(\sqrt{p} - 1) \end{aligned}$$

14.3 超立方上的 CT 播送

14.3.1 步骤

课件第 32 页给出：

依次从低维到高维播送
 $d = 0, 1, 2, 3, \dots$

14.3.2 通信时间

$$t_{\text{one-to-all}}(\text{CT}) = (t_s + m t_w) \log p$$

这里与 SF 模式下的超立方广播形式相同。

15. 8.4 多到多播送

多到多播送，也就是 all-to-all broadcast：

每个处理器都要把自己的消息发送给所有其他处理器。

这比一到多广播更复杂，因为网络中同时存在大量消息，容易发生链路竞争。

16. 多到多播送：SF 模式

16.1 环上的多到多播送

16.1.1 步骤

课件第 35 页图 8.10 展示环上的过程。

方法是：

所有处理器同时向右或向左播送刚接收到的信包

每一步，每个节点把自己新收到的信息继续传给下一个节点。

经过 $p-1$ 步，每个节点都收到所有其他节点的信息。

16.1.2 通信时间

$$t_{\text{all-to-all}}(\text{SF}) = (t_s + m t_w)(p - 1)$$

因为总共需要沿环传 $p-1$ 轮。

16.2 环绕网孔上的多到多播送

16.2.1 步骤

课件第 36 页图 8.11 给出方法：

1. 先进行的播送；
2. 再进行列的播送。

第一阶段，行内所有节点互相交换信息。
第二阶段，各列再交换已经汇总的行信息。

16.2.2 通信时间

课件给出：

$$\begin{aligned} t_{\text{all-to-all}}(\text{SF}) \\ = 2t_s(\sqrt{p} - 1) + m t_w(p - 1) \end{aligned}$$

含义是：

- 启动开销与行、列广播轮数有关；
- 数据传输总量最终与 $p-1$ 个其他处理器有关。

16.3 超立方上的多到多播送

16.3.1 步骤

课件第 37 页图 8.12 展示超立方上的多到多广播。

方法是：

依次按维进行多到多播送

每经过一维，节点掌握的信息数量翻倍。

例如 8 节点超立方：

第 1 维后：每个节点知道 2 个节点的信息
第 2 维后：每个节点知道 4 个节点的信息
第 3 维后：每个节点知道 8 个节点的信息

16.3.2 通信时间

课件给出：

$$\begin{aligned} t_{\text{all-to-all}}(\text{SF}) \\ = \sum_{i=1}^{\log p} (t_s + 2^{i-1} m t_w) \end{aligned}$$

化简为：

$$\begin{aligned} t_{\text{all-to-all}}(\text{SF}) \\ = t_s \log p + m t_w(p - 1) \end{aligned}$$

这里 $m t_w(p - 1)$ 表示每个处理器最终要接收其他 $p-1$ 个处理器的数据。

17. 多到多播送：CT 模式

17.1 链路竞争问题

课件第 39 页指出：

$$t_{\text{all-to-all}}(\text{CT}) = t_{\text{all-to-all}}(\text{SF})$$

原因是：

使用一到多的策略会造成链路竞争。

第 39 页图 8.13 展示了环上多条信包竞争同一通道的情况。

虽然 CT 对单条消息传输有优势，但在 all-to-all 中，大量消息同时存在，链路竞争成为主导因素，因此 CT 的优势会被削弱。

17.2 关键理解

这说明：

开关技术的优势不是在所有通信模式下都能体现。

对于单个长消息，CT 或虫孔可能比 SF 快很多。

但对于全交换、多到多通信，网络拥塞和链路竞争可能成为主要瓶颈。

18. 本章公式总表

18.1 单一信包一到一传输

网络	最大距离 l	SF 时间
一维环	$\lceil p/2 \rceil$	$t_s + t_w m \lceil p/2 \rceil$
环绕 Mesh	$2\lceil \sqrt{p}/2 \rceil$	$t_s + 2t_w m \lceil \sqrt{p}/2 \rceil$
超立方	$\log p$	$t_s + t_w m \log p$

CT 近似：

$$t_{\text{comm}}(\text{CT}) = t_s + m t_w$$

18.2 一到多播送 SF

网络	通信时间
环	$(t_s + m t_w) \lceil p/2 \rceil$
环绕 Mesh	$2(t_s + m t_w) \lceil \sqrt{p}/2 \rceil$
超立方	$(t_s + m t_w) \log p$

18.3 一到多播送 CT

网络	通信时间
环	$t_s \log p + m t_w \log p + t_h(p-1)$
网孔	$(t_s + m t_w) \log p + 2t_h(\sqrt{p}-1)$
超立方	$(t_s + m t_w) \log p$

18.4 多到多播送 SF

网络	通信时间
环	$(t_s + m t_w)(p-1)$
环绕 Mesh	$2t_s(\sqrt{p}-1) + m t_w(p-1)$
超立方	$t_s \log p + m t_w(p-1)$

CT 模式下课件给出：

$$t_{\text{all-to-all}}(\text{CT}) = t_{\text{all-to-all}}(\text{SF})$$

原因是链路竞争严重。

19. 本章核心总结

19.1 通信单位

消息 Message
包 Packet
片 Flit

消息是逻辑单位，包是传输单位，片是更细粒度的流水单位。

19.2 选路算法

本章重点掌握：

X-Y 选路：二维网孔，先 X 后 Y
E-Cube 选路：超立方，逐位翻转不同 bit

19.3 开关技术

重点掌握三种：

存储转发： $O(m \ l)$
切通： $O(m + \ l)$
虫孔：flit 流水传输，缓冲小，延迟低

19.4 基本通信操作

本章重点分析三类：

一到一传输
一到多播送
多到多播送

不同网络拓扑下，通信时间差异很大。

19.5 最重要的理解

这一章的主线是：

并行数值算法的性能不仅由计算量决定，还强烈受通信模式、网络拓扑、选路算法和开关技术影响。

尤其要记住：

局部通信通常容易扩展
全局通信容易成为瓶颈
超立方在广播类操作中通常效率较高
多到多通信容易发生链路竞争
CT 对单条消息有优势，但 `all-to-all` 中可能被竞争抵消

一句话概括：

第八章是在为后面的并行矩阵运算、线性方程求解和 FFT 做通信基础，因为这些数值算法的核心性能瓶颈往往不是“算不动”，而是“数据传不过去”。

PC9

1. 第九章总体：稠密矩阵运算

这份 **PC9.pdf** 讲第三篇“并行数值算法”中的 **第九章：稠密矩阵运算**。这一章主要围绕矩阵这种规则二维数据结构，讲如何在并行机上划分、转置、做矩阵-向量乘法，以及做矩阵乘法。

1.1 本章结构

第九章包括四节：

- 9.1 矩阵的划分
- 9.2 矩阵转置
- 9.3 矩阵-向量乘法
- 9.4 矩阵乘法

核心思想是：

稠密矩阵算法的并行性能，很大程度取决于矩阵如何划分、数据如何移动、通信是否均衡。

2. 9.1 矩阵的划分

矩阵划分是后面所有并行矩阵算法的基础。

2.1 为什么要划分矩阵

对于一个大矩阵 $A_{\{n \times n\}}$ ，单个处理器无法高效完成所有计算，所以需要把矩阵分配给多个处理器。

矩阵划分要解决三个问题：

- 每个处理器存哪部分数据？
- 每个处理器做哪部分计算？
- 处理器之间需要传哪些数据？

这章主要讲两类划分：

- 带状划分
- 棋盘划分

3. 9.1.1 带状划分

带状划分是把矩阵按行或按列切成条带。

3.1 列块带状划分

第 5 页图 9.1 左图展示了 16×16 矩阵在 $p=4$ 个处理器上的 **列块带状划分**。

矩阵按列连续切成 4 个竖条：

- P0 负责第 0~3 列
- P1 负责第 4~7 列
- P2 负责第 8~11 列
- P3 负责第 12~15 列

这种划分适合按列访问数据的算法。

3.2 行循环带状划分

第 5 页图 9.1 右图展示了 **行循环带状划分**。

不是连续分配行，而是按循环方式分配：

```
P0 负责第 0, 4, 8, 12 行
P1 负责第 1, 5, 9, 13 行
P2 负责第 2, 6, 10, 14 行
P3 负责第 3, 7, 11, 15 行
```

这种方式的好处是，如果每一行计算量不完全均匀，循环分配比连续块分配更容易负载均衡。

3.3 三种带状映射

第 6 页图中给出 `27x27` 矩阵在 `p=3` 个处理器上的三种带状划分：

```
block
cyclic
block-cyclic
```

3.3.1 Block 带状划分

连续大块分配。

```
P1 负责前 1/3 行
P2 负责中间 1/3 行
P3 负责后 1/3 行
```

优点是简单、局部性好。

缺点是负载可能不均衡。

3.3.2 Cyclic 带状划分

按行循环分配。

```
第 0 行给 P1
第 1 行给 P2
第 2 行给 P3
第 3 行再给 P1
...
```

优点是负载更均衡。

缺点是数据局部性差一些。

3.3.3 Block-Cyclic 带状划分

先把矩阵划成若干小块，再循环分配这些小块。

这是高性能数值库里很常见的方式，因为它兼顾：

```
block 的局部性
cyclic 的负载均衡
```

4. 9.1.2 棋盘划分

棋盘划分是把矩阵同时按行和列切成二维块。

4.1 块棋盘划分

第 8 页图 9.2 左图展示 `8x8` 矩阵在 `p=16` 个处理器上的块棋盘划分。

处理器排成 `4x4` 网格，每个处理器负责一个矩阵子块。

例如：

P0 负责左上角块
P1 负责第一行第二块
P4 负责第二行第一块
...

这种方式适合矩阵乘法、二维网格计算等二维结构问题。

4.2 循环棋盘划分

第 8 页图 9.2 右图展示循环棋盘划分。

它不是把连续区域给同一个处理器，而是把矩阵元素或小块按二维循环方式分配。

这样可以改善负载均衡，尤其当矩阵中不同位置的计算量不均匀时更有用。

4.3 三种棋盘映射

第 9 页图给出 16×16 矩阵在 $p=4$ ，也就是 2×2 处理器上的三种棋盘划分：

block
cyclic
block-cyclic

4.3.1 Block 棋盘划分

矩阵被分成四个大块，每个处理器一个大块。

4.3.2 Cyclic 棋盘划分

矩阵元素按棋盘格循环分配给处理器。

4.3.3 Block-Cyclic 棋盘划分

先划成小块，再循环分配小块。

现代并行线性代数程序常用 block-cyclic，因为它在缓存局部性和负载均衡之间比较折中。

5. 9.2 矩阵转置

矩阵转置是：

$$A[i,j] \rightarrow A[j,i]$$

在并行环境中，它本质上是一次数据重排。

5.1 为什么转置重要

矩阵转置经常出现在：

矩阵乘法
FFT
线性代数分解
数据布局转换

它的计算本身不复杂，但通信量可能很大。

6. 9.2.1 棋盘划分的矩阵转置

6.1 网孔连接， $p = n^2$ 情况

第 12 页图 9.3 展示了 $p=n^2$ 的情况。

这时每个处理器只存一个矩阵元素。

转置时，元素 (i,j) 要移动到 (j,i) 。

也就是：

$P(i, j)$ 中的数据发送到 $P(j, i)$

主对角线上的元素不用移动，因为：

$i = j$

图中左边是通信步，右边是转置后状态。可以看到，非对角线元素两两交换。

6.2 网孔连接， $p < n^2$ 情况

第 13 页图 9.4 展示 $p < n^2$ 的情况。

此时每个处理器存的是一个子块，而不是一个元素。

课件给出的算法是：

- ① 按 **mesh** 连接进行块转置，也就是不同处理器之间交换子块；
- ② 每个处理器内部进行块内转置。

也就是说，转置分成两层：

处理器间：交换块
处理器内：转置块内元素

如果矩阵 $A_{\{n \times n\}}$ 被划分成 p 个大小为：

$(n/\sqrt{p}) \times (n/\sqrt{p})$

的子块，那么每个处理器内部还要对这个子块做本地转置。

6.3 超立方连接上的棋盘转置

第 14、15 页讲超立方连接。

算法思想是递归转置。

把矩阵分成四块：

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

转置后变成：

$$A^T = \begin{bmatrix} A_{11}^T & A_{21}^T \\ A_{12}^T & A_{22}^T \end{bmatrix}$$

所以需要：

A_{12} 和 A_{21} 交换位置；
 A_{11} 、 A_{12} 、 A_{21} 、 A_{22} 各自内部继续递归转置。

第 15 页图展示了超立方连接下的交换路径。下方的超立方图说明，块之间的交换可以借助超立方维度递归完成。

课件给出的运行时间形式可以理解为：

通信递归步数约为 $\log p$
每次移动的数据块大小约为 n^2/p
再加上本地块内转置时间

所以它的核心是：

超立方通过递归维度交换，比较自然地支持矩阵转置。

7. 9.2.2 带状划分的矩阵转置

7.1 划分方式

第 17 页讲带状划分的转置。

矩阵 $A_{\{n \times n\}}$ 被分成 p 个行带：

每个处理器存放一个 $(n/p) \times n$ 的带

也就是：

P_0 存前 n/p 行
 P_1 存下一段 n/p 行
...

7.2 算法过程

转置时，每个处理器 P_i 需要把自己带中的数据再切成 p 个小块。

每个小块大小为：

$(n/p) \times (n/p)$

算法是：

- ① P_i 有 $p-1$ 个子块需要发送到其他 $p-1$ 个处理器；
- ② 每个处理器收到对应子块后，在本地交换相应元素。

第 17 页图 9.7 中的箭头表示不同处理器之间的子块交换。

7.3 关键理解

带状划分下，转置会导致大量跨处理器交换。

原因是：

转置前按行分布
转置后按列分布

如果仍然希望转置后的矩阵按行带分布，就必须进行全局数据重排。

8. 9.3 矩阵-向量乘法

矩阵-向量乘法是：

$y = A \times x$

其中：

$y_i = \sum_j a_{ij} x_j$

它是很多迭代算法的核心，例如 Jacobi、Gauss-Seidel、共轭梯度等。

9. 9.3.1 带状划分的矩阵-向量乘法

9.1 行带状划分

第 20 页讲行带状划分。

在 $p=n$ 的简单情形中：

P_i 存放 x_i 和第 i 行 $a_{i,0}, a_{i,1}, \dots, a_{i,n-1}$
 P_i 输出 y_i

也就是说，每个处理器负责计算一个 y_i 。

9.2 算法步骤

对于 $p=n$ ：

- ① 每个 P_i 向其他处理器播送 x_i ；
- ② 每个 P_i 计算自己的 y_i 。

为什么要播送 x_i ？

因为每一行计算 y_i 都需要整个向量 x ：

$$y_i = a_{i0} x_0 + a_{i1} x_1 + \dots + a_{i,n-1} x_{n-1}$$

所以每个处理器最终都必须得到完整的 x 。

9.3 $p < n$ 情况

如果 $p < n$ ，每个处理器负责 n/p 行，也对应存放 x 中的 n/p 个分量。

算法中， P_i 要播送 x 中相应的 n/p 个分量。

这本质上是一次 **多到多播送**。

9.4 超立方连接时间

课件第 20 页给出超立方连接下的计算时间。

可以理解为两部分：

前项：矩阵-向量乘法本地计算时间
后项：多到多播送时间

其主导形式为：

$$T_p \approx n^2/p + t_s \log p + n t_w$$

含义：

- n^2/p ：矩阵总共 n^2 次乘法，分给 p 个处理器；
- $t_s \log p$ ：超立方广播启动开销；
- $n t_w$ ：传输向量数据的代价。

9.5 网孔连接时间

网孔连接下，多到多通信比超立方更贵。

课件给出形式可理解为：

$$T_p \approx n^2/p + 2t_s(\sqrt{p} - 1) + n t_w$$

其中：

- $2t_s(\sqrt{p} - 1)$ 来自二维网孔中行、列方向的通信启动；

- `n t_w` 是传输整个向量数据的代价。

9.6 第 21 页图 9.8 的过程

图 9.8 展示了带状划分矩阵-向量乘法的四个状态：

- (a) 矩阵 `A` 按行带分布
- (b) 向量 `x` 的分量沿处理器传播
- (c) 每个处理器获得完整 `x` 后进行局部计算
- (d) 输出向量 `y` 按行结果分布

这张图的重点是：

行带状划分下，矩阵分布很自然，但向量 `x` 必须广播给所有处理器。

10. 9.3.2 棋盘划分的矩阵-向量乘法

10.1 块棋盘划分

第 23 页讲棋盘划分。

处理器排成二维网孔：

$$\sqrt{p} \times \sqrt{p}$$

每个处理器 `Pij` 存放矩阵子块 `Aij`。

向量 `x` 的相应部分放入对角线处理器：

`x_i` 置入 `Pii` 中

10.2 $p = n^2$ 情况

在最细粒度情况下，每个处理器存一个矩阵元素。

算法：

- ① 每个 `Pii` 向 `Pj,i` 播送 `xi`；
- ② 按行方向进行乘-加与积累；
- ③ 最后一列 `Pi,n-1` 收集的结果为 `yi`。

也就是说：

先按列方向传播 `x`
再按行方向累加 `aij*xj`

10.3 $p < n^2$ 情况

对于一般情况，`p` 个处理器排成 $\sqrt{p} \times \sqrt{p}$ 的二维网孔。

每个处理器负责一个矩阵块。

`Pii` 向对应列播送 `x` 中的 $n/\sqrt{p}$ 个分量。

然后每一行进行局部乘加和结果累积。

10.4 网孔连接计算时间

课件第 23 页给出 CT 模式下时间分析，主要包括三部分：

- 1. `x` 的相应分量置入对角处理器的通信时间
- 2. 按列一到多播送时间
- 3. 按行单点积累时间

最终近似形式为：

$$T_p \approx n^2/p + t_s \log p + (n/\sqrt{p})t_w \log p + 3t_h\sqrt{p}$$

可以看出，棋盘划分降低了向量通信压力，因为每次只传播 $n/\sqrt{p}$ 的向量块，而不是整个 n 长向量。

10.5 第 24 页图 9.9 的过程

图 9.9 展示棋盘划分矩阵-向量乘法过程：

- (a) 矩阵 A 按二维块分布，向量 x 分块放在对应位置
 - (b) x 沿列方向播送
 - (c) 各处理器局部乘法
 - (d) 沿行方向积累，得到 y

这体现了二维划分的优势：

数据传播从“一维全局广播”变成“二维方向上的局部广播和归约”。

11. 带状划分与棋盘划分比较

第 25 页直接比较了二者。

11.1 网孔上带状划分运行时间

课件给出：

$$T_p = n^2/p + 2t_s(\sqrt{p} - 1) + n t_w$$

11.2 网孔上棋盘划分运行时间

课件给出：

$$T_p \approx n^2/p + t_s \log p + (n/\sqrt{p})t_w \log p + 3t_h\sqrt{p}$$

11.3 结论

课件明确说：

棋盘划分要比带状划分快。

原因是：

- 带状划分需要广播整个向量 x ；
 - 棋盘划分只需要在二维网格内传播向量块。

所以对于大规模矩阵-向量乘法，二维棋盘划分通常更可扩展。

12. 9.4 矩阵乘法

矩阵乘法是本章重点。

目标是：

$$C = A \times B$$

其中：

$$c_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$$

第 28 页图强调了矩阵乘法的几何意义：

C 的第 i 行第 j 列元素
=
A 的第 i 行
与
B 的第 j 列
做内积

也就是：

A 中元素的第 1 下标与 B 中元素的第 2 下标对准

这里的“对准”很重要，因为后面 Cannon、Fox、Systolic、DNS 的核心都是让需要相乘的 A 块和 B 块在正确时间或正确位置相遇。

13. 矩阵乘法并行实现方法

第 29 页把矩阵乘法分成两类实现思想。

13.1 空间对准

空间对准指：

需要相乘的矩阵元素或矩阵块已经被加载到同一个处理器阵列位置上。

典型算法包括：

Cannon
Fox
DNS

它们强调把 A 和 B 的数据移动到合适处理器上，再进行局部乘加。

13.2 时间对准

时间对准指：

元素一开始不一定在同一处理器中，而是在计算过程中按时间顺序流入处理器并相遇。

典型算法是：

Systolic

也就是脉动阵列乘法。

它强调数据像流水一样进入阵列，在正确的时间点进行乘加。

14. 9.4.1 简单并行分块乘法

14.1 分块方式

把 A、B、C 都分成：

$\sqrt{p} \times \sqrt{p}$

个方块矩阵。

每个块大小为：

$(n/\sqrt{p}) \times (n/\sqrt{p})$

处理器也编号为：

P00, P01, ..., P $\sqrt{p}-1$, P $\sqrt{p}-1$

每个处理器 Pij 存放：

A_{ij} , B_{ij} , C_{ij}

14.2 算法步骤

第 30 页给出的算法是：

- ① 通信：
每行处理器进行 A 矩阵块的多到多播送；
每列处理器进行 B 矩阵块的多到多播送。
- ② 乘-加运算：
 P_{ij} 计算：
 $C_{ij} = \sum_k A_{ik} B_{kj}$

也就是说， P_{ij} 最终要得到本行所有 A_{ik} 和本列所有 B_{kj} ，才能算出 C_{ij} 。

14.3 超立方连接时间

课件给出通信和计算两部分：

通信时间：
 $2(t_s \log \sqrt{p} + t_w(n^2/p)(\sqrt{p} - 1))$

计算时间：
 n^3/p

所以总时间主导为：

$$T_p = n^3/p + \text{通信项}$$

14.4 二维环绕网孔连接时间

课件第 31 页给出二维环绕网孔下：

$$\begin{aligned} t_1 &= 2t_s\sqrt{p} + 2t_w n^2/\sqrt{p} \\ t_2 &= n^3/p \end{aligned}$$

因此：

$$T_p = n^3/p + 2t_s\sqrt{p} + 2t_w n^2/\sqrt{p}$$

14.5 缺点

课件特别指出简单分块算法的缺点：

对处理器的存储要求过大。

每个处理器需要存多个块，整体存储开销比串行算法更大。

当 $p=n^2$ 时：

$$\begin{aligned} t(n) &= O(n) \\ c(n) &= O(n^3) \end{aligned}$$

这说明它理论上可以很快，但存储和通信代价不一定理想。

15. 9.4.2 Cannon 乘法

15.1 分块方式

Cannon 乘法和简单分块一样，把矩阵划成：

$$\sqrt{p} \times \sqrt{p}$$

个块，每个块大小是：

$$(n/\sqrt{p}) \times (n/\sqrt{p})$$

处理器 P_{ij} 存放：

$$A_{ij}, B_{ij}, C_{ij}$$

第 33 页图展示了 4×4 处理器网格上的分块布局。

15.2 算法核心思想

Cannon 的核心是 **循环移位对齐**。

第 34 页给出非形式算法：

- ① A_{ij} 向左循环移动 i 步；
 B_{ij} 向上循环移动 j 步；
 - ② 每个处理器 P_{ij} 做一次 A_{ij} 和 B_{ij} 的乘-加；
 - ③ A 的每个块向左循环移动一步；
 B 的每个块向上循环移动一步；
 - ④ 重复执行 $\sqrt{p}$ 次。

这样每个处理器最终会遇到所有需要相乘的块。

15.3 初始对齐

第 35 页图展示 $A_{4 \times 4}$ 、 $B_{4 \times 4}$ 、 $p=16$ 时的初始对齐：

$$\begin{matrix} A \text{ 的第 } i \text{ 行左移 } i \text{ 步} \\ B \text{ 的第 } j \text{ 列上移 } j \text{ 步} \end{matrix}$$

例如：

$$\begin{matrix} A \text{ 第 } 0 \text{ 行不动} \\ A \text{ 第 } 1 \text{ 行左移 } 1 \text{ 步} \\ A \text{ 第 } 2 \text{ 行左移 } 2 \text{ 步} \\ A \text{ 第 } 3 \text{ 行左移 } 3 \text{ 步} \end{matrix}$$

B 的列也类似上移。

15.4 每轮循环移位

第 36、37 页展示初始对齐后，每一步继续移位的状态。

每一轮：

$$\begin{matrix} \text{本地块相乘并累加到 } C_{ij} \\ A \text{ 块左移一步} \\ B \text{ 块上移一步} \end{matrix}$$

经过 $\sqrt{p}$ 轮后，每个处理器都完成自己的 C_{ij} 块。

15.5 Cannon 形式化算法

第 38 页给出 Cannon 分块乘法算法。

整体分三步：

1. 初始对齐：
A 行循环左移；
B 列循环上移。

2. 初始化：
Cij = 0。

3. 循环 $\sqrt{p}$ 次：
Cij = Cij + Aij Bij；
Aij 左移一步；
Bij 上移一步。

15.6 时间复杂度

课件第 38 页给出：

$$\begin{aligned} T_p(n) &= T_1 + T_2 + T_3 \\ &= O(\sqrt{p}) + O(1) + O(\sqrt{p} \cdot (n/\sqrt{p})^3) \\ &= O(n^3/p) \end{aligned}$$

这里：

- 初始对齐需要 $O(\sqrt{p})$ 移位；
- 初始化是 $O(1)$ ；
- 每轮局部块乘法规模为 $(n/\sqrt{p})^3$ ，共 $\sqrt{p}$ 轮；
- 总计算主项为 $O(n^3/p)$ 。

Cannon 的优点是：

存储需求比简单分块小
通信规则简单
适合二维环绕网络
负载均衡好

16. 9.4.3 Fox 乘法

16.1 分块方式

Fox 乘法的分块和 Cannon 相同。

也就是矩阵分成 $\sqrt{p} \times \sqrt{p}$ 个块，处理器排成二维网格。

16.2 算法原理

第 40 页给出 Fox 乘法原理：

① $A_{i,i}$ 向所在行的其他处理器进行一到多播送；

② 各处理器将收到的 A 块与原有 B 块进行乘-加；

③ B 块向上循环移动一步；

④ 下一轮选择 $A_{i,(j+1) \bmod \sqrt{p}}$ 进行行播送；

⑤ 重复执行 $\sqrt{p}$ 次。

更直观地说，Fox 和 Cannon 的区别是：

Cannon: A 和 B 都通过循环移位逐步对齐；
Fox: 每轮在行内广播一个 A 块，B 块上移。

16.3 计算过程

在第 k 轮中, 第 i 行选择:

```
A_i, (i+k) mod √p
```

作为本轮要广播的 A 块。

这一块在第 i 行广播给所有处理器。

每个处理器收到 A 块后, 与当前本地 B 块相乘并累加到 C。

然后 B 块上移一步, 为下一轮做准备。

16.4 第 41、42 页示例

第 41、42 页展示 $A_{4 \times 4}$ 、 $B_{4 \times 4}$ 、 $p=16$ 时 Fox 乘法的 4 个阶段。

可以看到:

```
第 1 轮: 各行广播对角块 A00, A11, A22, A33;
第 2 轮: 各行广播下一块 A01, A12, A23, A30;
第 3 轮: 继续广播 A02, A13, A20, A31;
第 4 轮: 广播 A03, A10, A21, A32。
```

同时 B 块每轮向上循环移动。

16.5 Fox 与 Cannon 对比

算法	A 的移动方式	B 的移动方式	特点
Cannon	A 每轮左移	B 每轮上移	全靠循环移位对齐
Fox	每轮行广播 A	B 每轮上移	用广播减少 A 的逐步移动

Fox 更依赖行广播能力; Cannon 更适合规则二维环绕网格。

17. 9.4.4 Systolic 乘法

Systolic 是脉动阵列乘法。

17.1 基本思想

Systolic 乘法属于 时间对准。

也就是说, A 和 B 的元素不是一开始都放在正确位置, 而是随着时间流入处理器阵列。

每个处理器 P_{ij} 负责计算一个结果元素:

```
c_ij
```

当一个 a 和一个 b 同时到达 P_{ij} 时, 处理器执行:

```
c_ij = c_ij + a × b
```

然后:

```
a 向右传
b 向下传
```

17.2 第 44 到 54 页图示

第 44 至 54 页连续展示了 Systolic 乘法的时间步。

图中 **A** 的元素从左侧进入阵列，**B** 的元素从上方进入阵列。

每一步中，某些 a_{ik} 和 b_{kj} 在处理器 P_{ij} 相遇，并累加到 c_{ij} 。

例如：

$$c_{1,1} = a_{1,1} b_{1,1} + a_{1,2} b_{2,1} + a_{1,3} b_{3,1} + a_{1,4} b_{4,1}$$

第 54 页显示算法结束后，每个处理器中保存对应的 c_{ij} 。

17.3 Systolic 算法描述

第 55 页给出算法：

```
输入: A_{m \times n}, B_{n \times k}
输出: C_{m \times k}

for i = 1 to m par-do
  for j = 1 to k par-do
    c_{i,j} = 0
    while P_{i,j} 收到 a 和 b 时 do
      c_{i,j} = c_{i,j} + a b
      if i < m then 发送 b 给 P_{i+1,j}
      if j < k then 发送 a 给 P_{i,j+1}
    endwhile
  endfor
endfor
```

17.4 直观理解

每个处理器像一个小计算单元：

左边进 **A**
上边进 **B**
本地做乘加
A 往右传
B 往下传

所以整个矩阵乘法像波一样在处理器阵列中推进。

17.5 Systolic 的优点

通信局部，只和相邻处理器通信
结构规则，适合硬件实现
数据流动有节奏，吞吐率高
适合 **VLSI** 或专用阵列处理器

17.6 Systolic 的缺点

启动和排空流水线需要时间
数据输入要按特定时间错开
对规则矩阵乘法很适合，对不规则问题不灵活

18. 9.4.5 DNS 乘法

DNS 是 Dekel、Nassimi 和 Sahni 提出的并行矩阵乘法算法。

18.1 背景

第 57 页说，DNS 是 SIMD-CC 上的矩阵乘法算法。

它使用：

n^3 个处理器

运行时间为：

$O(\log n)$

是一种非常快但处理器需求极高的算法。

18.2 基本思想

矩阵乘法中：

$c_{ij} = \sum_k a_{ik} b_{kj}$

DNS 的思想是：

每个处理器负责一个乘积 $a_{ik} b_{kj}$
然后沿 k 方向把这些乘积求和

处理器用三维坐标编号：

(k, i, j)

让处理器 (k, i, j) 拥有：

$a_{i,k}$
 $b_{k,j}$

然后本地计算：

$a_{i,k} \times b_{k,j}$

最后沿 k 维度求和，结果存到：

$(0, i, j)$

18.3 处理器编号

课件给出处理器数：

$p = n^3 = (2^q)^3 = 2^{3q}$

处理器 p_r 位于位置：

(k, i, j)

其中：

$r = k n^2 + i n + j$

并且：

$$0 \leq i, j, k \leq n-1$$

每个处理器有三个寄存器：

```
A[k,i,j]
B[k,i,j]
C[k,i,j]
```

初始时都为 0。

18.4 DNS 算法步骤

第 57、59 页给出 DNS 算法思想和形式描述。

主要分三步。

18.4.1 数据复制

初始时：

```
a_i,j  存在 A[0,i,j]
b_i,j  存在 B[0,i,j]
```

然后：

```
A、B  同时在 k 维复制；
A 在 j 维复制；
B 在 i 维复制。
```

目的就是让每个处理器 (k, i, j) 同时拿到：

```
a_i,k
b_k,j
```

18.4.2 本地相乘

所有处理器并行执行：

$$C[k,i,j] = A[k,i,j] \times B[k,i,j]$$

这一步完全并行。

18.4.3 沿 k 方向求和

对固定的 (i, j) ，沿 k 方向把所有乘积求和：

$$C[0,i,j] = \sum_k C[k,i,j]$$

这就是最终矩阵元素：

```
c_ij
```

18.5 DNS 示例

第 58 页展示 2×2 的小例子。

第 60、61 页展示更一般的三维结构：

图中用 k 维表示不同层，每一层是一个 $i-j$ 平面。A 和 B 被复制到三维阵列中，最后沿 k 方向求和。

这张图的重点是：

DNS 把矩阵乘法的三重循环 i, j, k 全部空间化，用三维处理器阵列直接对应三重下标。

18.6 DNS 的优缺点

18.6.1 优点

并行度极高

运行时间可达 $O(\log n)$

结构清楚

适合理论分析

18.6.2 缺点

需要 n^2 个处理器

存储和数据复制开销很大

实际机器上成本极高

所以 DNS 更像理论上的高并行算法，说明如果硬件资源极其充足，矩阵乘法可以被压缩到很短时间。

19. 本章算法总对比

内容	核心思想	通信特点	适用场景
带状划分	按行或列分条带	一维广播较多	简单矩阵-向量乘
棋盘划分	按二维块分布	通信更局部	大规模矩阵运算
矩阵转置	<code>A[i,j] → A[j,i]</code>	数据重排明显	FFT、矩阵重布局
带状矩阵-向量乘	每行负责一个输出	需要广播 x	处理器较少或简单实现
棋盘矩阵-向量乘	二维分块	列播送 + 行归约	更可扩展
简单分块乘法	行列块广播	存储需求大	理论入门
Cannon	循环移位对齐	邻居通信	二维环绕网格
Fox	行广播 A，列移位 B	行广播较多	适合广播效率高的网格
Systolic	数据流时间对准	邻居流水通信	专用阵列、规则计算
DNS	三维处理器阵列	大量复制 + k 维归约	理论高速算法

20. 本章核心总结

第九章的主线是：

矩阵如何划分

→ 数据如何重排

→ 矩阵-向量乘法如何通信

→ 矩阵乘法如何对准 **A** 和 **B**

最重要的理解是：

稠密矩阵运算的计算结构非常规则，但通信代价决定了并行算法是否高效。

20.1 复习重点

需要重点掌握：

1. 带状划分、棋盘划分、`block-cyclic` 的区别

2. 矩阵转置为什么会致全局数据重排

3. 带状矩阵-向量乘法为什么要广播整个 x

4. 棋盘矩阵-向量乘法为什么通常比带状划分快

5. `Cannon` 的初始对齐和循环移位

6. `Fox` 的行广播和 **B** 上移

7. `Systolic` 的时间对准和邻居流水传输

8. `DNS` 的三维处理器阵列和 k 维归约

一句话概括：

第九章讲的是并行稠密矩阵运算的“数据布局 + 通信模式 + 计算对准”，其中 Cannon、Fox、Systolic、DNS 分别代表了二维移位、二维广播、流水阵列和三维超并行四种不同的矩阵乘法思想。